

[LOGO UNIVERSITÉ / ÉCOLE] [LOGO ENTREPRISE / LABORATOIRE]

CONCEPTION ET DÉVELOPPEMENT D'UNE PLATEFORME INTELLIGENTE ET SÉCURISÉE DE COMPRESSION DE DONNÉES BASÉE SUR L'INTELLIGENCE ARTIFICIELLE

XAI-COMPRESS

Mémoire de projet de fin d'études
[ÉTABLISSEMENT] — [FORMATION]
Auteur : [À COMPLÉTER]
Encadrant(s) : [À COMPLÉTER]
Année universitaire : [À COMPLÉTER]

Dédicaces

[À COMPLÉTER — texte personnel facultatif]

Remerciements

[À COMPLÉTER — établissement, encadrants, équipe et jury]

Résumé

Ce mémoire présente XAI-Compress, une plate-forme multi-client de compression sans perte associant sélection hybride de codecs, API FastAPI, clients Next.js, Angular, Flutter et .NET, PostgreSQL et contrôles ClamAV/YARA. L'étude montre que Hybrid V3 top-3 améliore fortement le débit face à Brotli-11 sur 140 fichiers, tout en restant 0,7591 % plus volumineux. L'apport majeur est l'intégration vérifiable du routage adaptatif, de l'intégrité SHA-256, du MFA TOTP et du fail-closed.

Abstract

This report presents XAI-Compress, a multi-client lossless-compression platform combining hybrid codec selection, FastAPI, Next.js, Angular, Flutter, .NET, PostgreSQL, and ClamAV/YARA controls. Hybrid V3 top-3 greatly improves compression throughput over Brotli-11 on 140 held-out files while producing a 0.7591% larger artifact.

Table des matières — structure

- 0. Introduction générale
- 1. Chapitre 1 — Contexte et état de l'art
- 2. Chapitre 2 — Analyse et spécification des besoins
- 3. Chapitre 3 — Architecture globale
- 4. Chapitre 4 — Backend FastAPI
- 5. Chapitre 5 — Base de données
- 6. Chapitre 6 — Moteur de compression IA
- 7. Chapitre 7 — Méthodologie d'entraînement
- 8. Chapitre 8 — Architecture de sécurité
- 9. Chapitre 9 — Authentificateur mobile Flutter
- 10. Chapitre 10 — Application Desktop Flutter
- 11. Chapitre 11 — Portail Angular
- 12. Chapitre 12 — Site public Next.js
- 13. Chapitre 13 — Administration .NET
- 14. Chapitre 14 — E-mail et cycle de compte
- 15. Chapitre 15 — Conteneurisation et DevOps
- 16. Chapitre 16 — Tests et assurance qualité
- 17. Chapitre 17 — Performances et résultats
- 18. Chapitre 18 — Défis, limites et perspectives

Liste des figures et tableaux

12 figures techniques générées; 11 tableaux principaux et annexes détaillées.

Liste des acronymes

Acronyme	Critère
Application Programming Interface	
Application Byte	
Recurrent Unit	
Web Token	
Factor Authentication	
based One-Time Password	
neur XAI-Compress	

Introduction générale

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Contexte et problématique

La croissance des données accentue simultanément les coûts de stockage, le temps de transfert et la surface d'attaque liée au traitement des fichiers.

Une plate-forme de compression moderne ne peut donc être évaluée sur le seul taux de réduction: elle doit préserver exactement les octets, borner les ressources, contrôler les contenus hostiles et offrir un parcours cohérent sur plusieurs clients.

La problématique retenue est la suivante: comment concevoir une plate-forme multi-client capable de sélectionner une stratégie de compression sans perte selon les caractéristiques d'un fichier, tout en assurant intégrité, authentification forte, analyse antimalware et traçabilité, avec des résultats reproductibles et sans transformer une meilleure vitesse en prétention infondée de meilleure compression?

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Objectifs et méthode

XAI-Compress articule un site public Next.js, un portail Angular, un client Flutter Desktop, un authenticateur Flutter, une administration Blazor .NET et une API FastAPI.

L'audit suit une méthode de preuve: lecture du code, rapprochement des configurations, inspection des migrations, analyse des tests et confrontation aux rapports de benchmark.

Les faits sont marqués comme vérifiés, historiques, proposés ou manquants.

Cette discipline est indispensable lorsque l'architecture déclarée excède ce qui a été validé en production.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Organisation du mémoire

Les chapitres progressent de l'état de l'art et des exigences vers l'architecture, l'implémentation, l'IA, la sécurité et les clients.

Ils se terminent par le DevOps, les tests, les résultats, les difficultés et les perspectives.

Les annexes consolident les endpoints, variables sans valeurs, tables, preuves et captures à produire.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Conclusion du chapitre

Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Chapitre 1 — Contexte et état de l'art

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Fondements de la compression

La compression sans perte exploite la redondance statistique sans modifier le message reconstruit.

Huffman affecte des mots courts aux symboles fréquents; LZ77 et LZ78 décrivent des répétitions par références; LZW construit un dictionnaire; DEFLATE associe LZ77 et Huffman; gzip et ZIP ajoutent des conteneurs et métadonnées; LZMA recherche des ratios élevés au prix d'un calcul plus important.

Aucun algorithme ne domine universellement: les médias déjà compressés, les petits fichiers et les données quasi aléatoires déplacent le compromis entre en-tête, ratio, mémoire et latence.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Modélisation neuronale et codage entropique

Un modèle causal byte-level estime $p(x_t|x_{<t})$ sur 256 valeurs.

La cross-entropie moyenne, exprimée en bits par octet, approche le coût idéal d'un code entropique, mais le conteneur réel ajoute modèle, métadonnées, checksums et alignement.

La GRU utilise une porte de mise à jour z_t , une porte de réinitialisation r_t et un état h_t ; elle réduit le problème de gradient par rapport au RNN simple, avec moins de paramètres qu'une LSTM.

La décompression impose exactement le même modèle, le même checkpoint et le même ordre numérique.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Positionnement XAI

Le nom XAI-Compress ne correspond pas, dans le code audité, à une méthode complète d'explicabilité locale de type SHAP appliquée à chaque octet.

La dimension explicable réside surtout dans le sélecteur de codecs, les caractéristiques observables, les routes top-3, les rapports d'ablation et la traçabilité des décisions.

Cette interprétation limitée est plus fidèle que l'affirmation d'une IA intrinsèquement explicable.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Comparaison critique

Les solutions classiques sont matures, portables et souvent plus compactes.

L'approche hybride vise surtout un routage adaptatif et un meilleur compromis de débit.

Les mesures V3 montrent précisément cette nuance: Hybrid V3 est beaucoup plus rapide que Brotli-11 sur le corpus retenu, mais produit un artefact légèrement plus grand.

La contribution scientifique tient donc à la sélection contrôlée, au protocole de comparaison et à l'intégration sécurisée, non à une victoire universelle sur Brotli.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Tableau 1 — Technologies

Couche	Technologie	Rôle	Limite
Public	Next.js	Présentation/statut	Pas de logique d'autorisation
Portail	Angular	Parcours fichiers	Tokens côté navigateur
Desktop	Flutter	Local et cloud	Dépendances moteur local
Authenticator	Flutter	TOTP/biométrie	Validation appareil requise

<i>Couche</i>	<i>Technologie</i>	<i>Rôle</i>	<i>Limite</i>
Admin	Blazor .NET	Lecture opérationnelle	Peu de tests dédiés
API	FastAPI	Contrats/orchestration	Traitement synchrone
Données	PostgreSQL	Persistance	Rétention non finalisée
Sécurité	ClamAV/YARA	Analyse	Disponibilité/signatures

Interprétation — La synthèse est circonscrite aux preuves indiquées.

Conclusion du chapitre

Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Chapitre 2 — Analyse et spécification des besoins

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Acteurs et responsabilités

L'utilisateur standard crée un compte, vérifie son adresse, active éventuellement le MFA, compresse, décompresse, télécharge, consulte l'historique et partage.

L'administrateur observe utilisateurs, fichiers, scanner et événements sans qu'une interface visible ne remplace l'autorisation serveur.

L'authentificateur mobile provisionne et protège le facteur TOTP.

Les services externes comprennent la messagerie, PostgreSQL et les scanners.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Exigences fonctionnelles

Les préconditions critiques sont l'identité validée, l'autorisation sur la ressource, la taille admissible et la disponibilité des scanners.

Chaque opération produit un résultat vérifiable ou une erreur non ambiguë.

Un partage est lié à un destinataire, une expiration, un nombre de téléchargements et une révocation.

La décompression ne doit jamais publier un contenu avant validation du conteneur et contrôle antimalware.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Exigences non fonctionnelles

La confidentialité des secrets, l'intégrité SHA-256, le fail-closed, les limites de ressources, l'audit, la maintenabilité par composants, la portabilité des clients et la reproductibilité des benchmarks forment les exigences structurantes.

La disponibilité n'autorise pas l'abaissement silencieux des contrôles.

La scalabilité est aujourd'hui limitée par le traitement local et le stockage configuré; l'objet storage et la file distribuée restent des perspectives.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Tableau 2 — Exigences fonctionnelles

ID	Exigence	Acteur	Priorité	Résultat attendu
F-01	Créer et vérifier un compte	Utilisateur	Haute	Compte activé sans divulgation de code
F-02	S'authentifier avec MFA	Utilisateur	Haute	Session et tokens bornés
F-03	Compresser un fichier scanné	Utilisateur	Critique	XAIC + SHA + historique
F-04	Décompresser sous limites	Utilisateur	Critique	Octets identiques ou rejet
F-05	Partager à un destinataire	Utilisateur	Haute	Code haché, expiration, quota
F-06	Observer les événements	Admin	Haute	Vue autorisée et assainie

Interprétation — La synthèse est circonscrite aux preuves indiquées.

Tableau 3 — Exigences non fonctionnelles

ID	Catégorie	Critère
NF-01	Sécurité	Échec fermé si scanner indisponible
NF-02	Intégrité	SHA-256 de round-trip
NF-03	Performance	Temps bout-en-bout mesuré
NF-04	Maintenabilité	Clients découplés du moteur
NF-05	Vie privée	Aucun secret dans logs/rapport
NF-06	Portabilité	Web, Windows et Android configurables

Interprétation — La synthèse est circonscrite aux preuves indiquées.

Conclusion du chapitre

Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Chapitre 3 — Architecture globale

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Architecture logique

Tous les clients communiquent avec FastAPI par REST.

L'API concentre validation, identité, autorisation et orchestration; SQLAlchemy isole la persistance; le moteur est invoqué derrière une frontière de service; les scanners forment une barrière avant et après transformation.

Ce découplage évite que les clients manipulent des chemins serveur ou des secrets de base de données.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Flux et frontières de confiance

Le poste client, Internet, la terminaison TLS, l'API, le réseau Docker, la base et le stockage sont des zones distinctes.

Un nom de fichier et un MIME sont non fiables; l'identité portée par un JWT est vérifiée mais l'autorisation doit encore contrôler le propriétaire.

Les sorties du moteur ne deviennent pas sûres par leur seule provenance: elles sont bornées, vérifiées et rescannées.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Architecture physique et déploiement

Les configurations prévoient Vercel pour les frontends et Render pour les services applicatifs, tandis que Compose assemble le développement avec PostgreSQL, ClamAV et Mailpit.

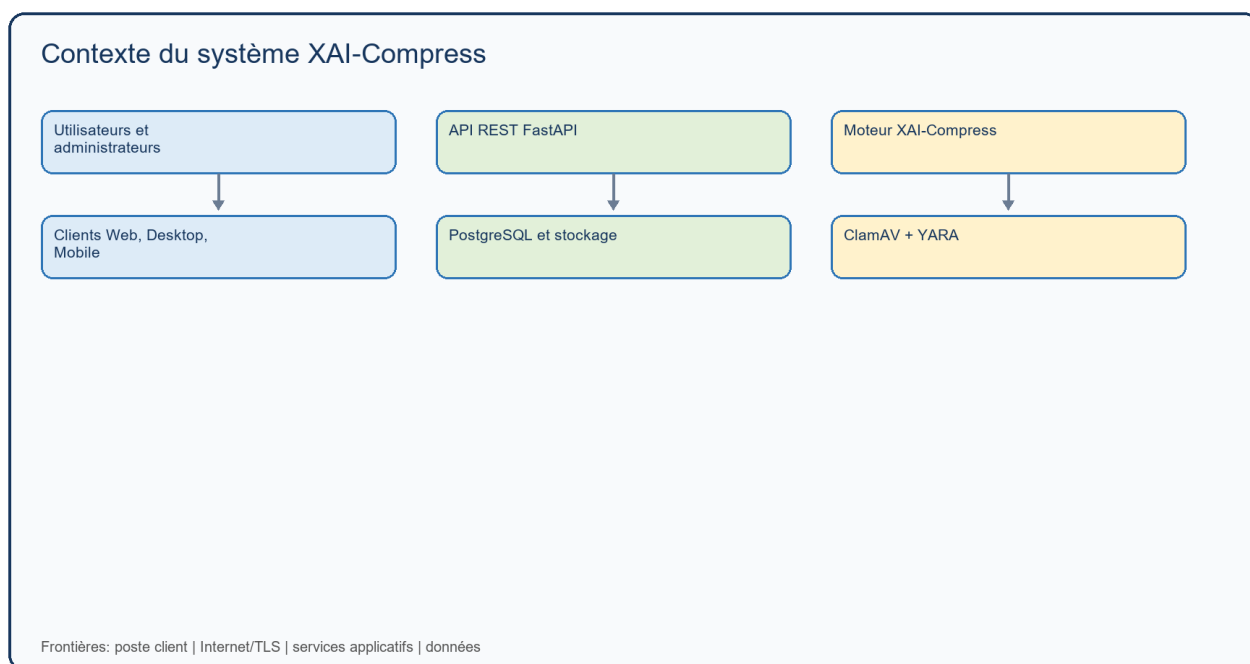
Ce rapport distingue la topologie définie du déploiement effectivement prouvé: aucune disponibilité de production n'est déduite d'un fichier de configuration.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Conclusion du chapitre

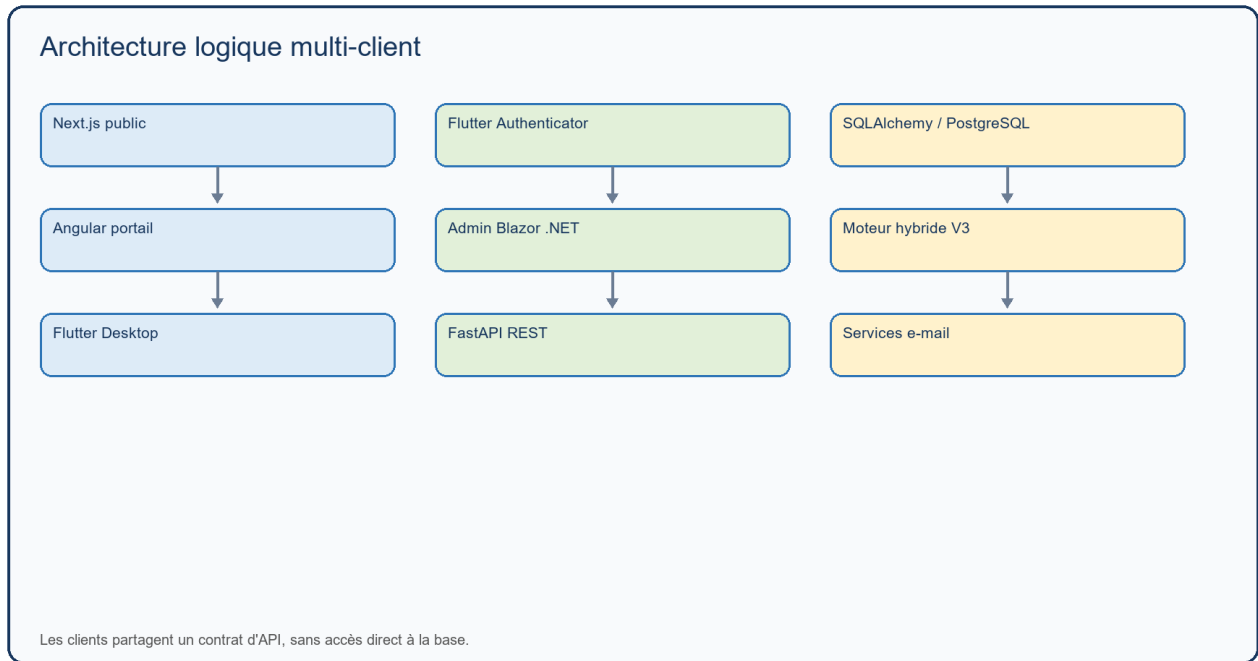
Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Figure 1 — System context



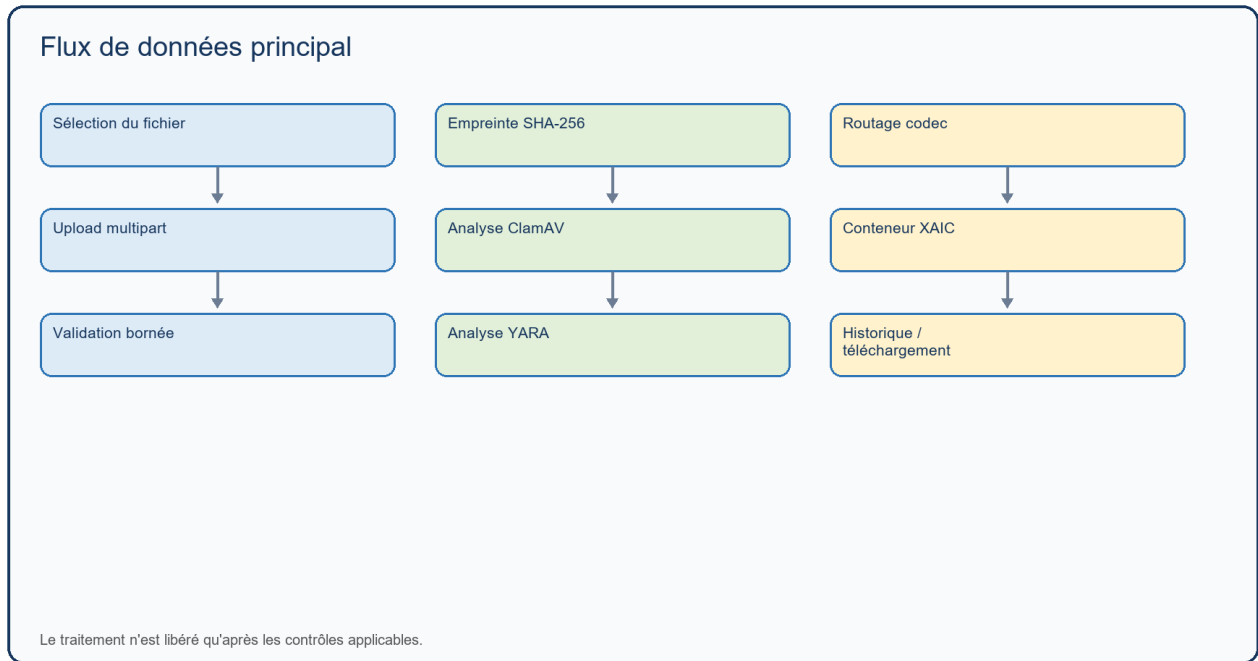
Interprétation — Le diagramme matérialise les responsabilités, dépendances et frontières où les contrôles doivent intervenir.

Figure 2 — Component architecture



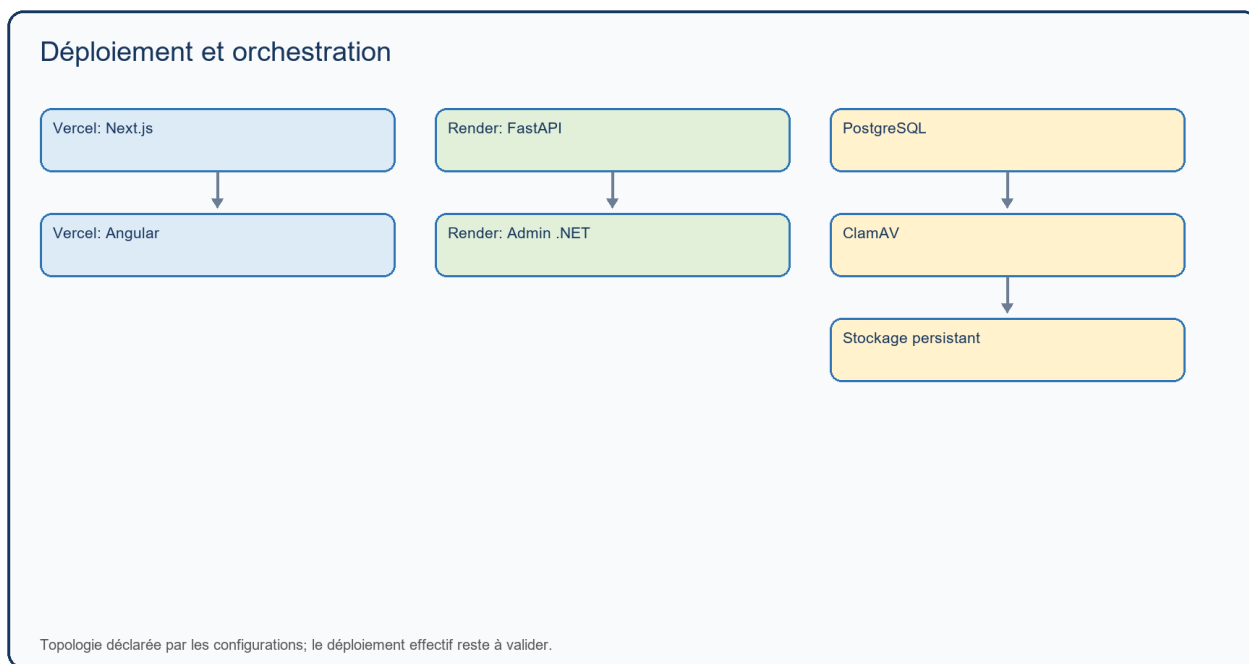
Interprétation — Le diagramme matérialise les responsabilités, dépendances et frontières où les contrôles doivent intervenir.

Figure 3 — Data flow



Interprétation — Le diagramme matérialise les responsabilités, dépendances et frontières où les contrôles doivent intervenir.

Figure 4 — Deployment



Interprétation — Le diagramme matérialise les responsabilités, dépendances et frontières où les contrôles doivent intervenir.

Chapitre 4 — Backend FastAPI

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Organisation et contrats REST

Les modules `main.py`, `account.py`, `mfa.py` et `authenticator.py` enregistrent 43 routes observées.

Pydantic valide les entrées; les dépendances FastAPI extraient l'utilisateur; SQLAlchemy gère les unités de travail.

Les réponses publiques d'état sont volontairement assainies.

CORS dépend d'une liste d'origines et les tests couvrent le rejet d'une origine non autorisée.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Cycle d'identité

L'inscription hache le mot de passe et crée un compte non vérifié.

La vérification e-mail utilise un code haché, borné en tentatives et en expiration.

La connexion vérifie les identifiants puis le MFA selon la politique.

Le JWT d'accès est court; le refresh token n'est persisté que sous forme de hachage et peut être révoqué.

L'oubli de mot de passe sépare demande, vérification de code et remplacement, ce qui réduit la durée d'exposition d'un jeton de réinitialisation.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Traitement de fichiers et erreurs

Le téléversement est copié sous limites, haché, scanné puis transmis au moteur.

Les erreurs internes sont traduites sans exposer chemins ni commandes.

L'historique est filtré par propriétaire; l'administration utilise un contrôle de rôle.

Les migrations 003/004 et leur runner sont testés, notamment pour maintenir l'alignement de `audit_events` et de son `updated_at`.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Tableau 4 — API principale

Méthode	Endpoint	Auth.	But
POST	/auth/register	Non	Créer un compte
POST	/auth/login	Non/MFA	Ouvrir une session
POST	/auth/refresh	Refresh	Renouveler
POST	/compression/jobs	JWT	Compresser
POST	/compression/decompress	JWT	Décompresser
GET	/history	JWT	Historique propriétaire
POST	/shares	JWT	Créer partage
POST	/shares/redeem	JWT	Racheter partage
GET	/admin/audit	Admin	Audit
GET	/public/status	Public	État assaini

Interprétation — La synthèse est circonscrite aux preuves indiquées.

Conclusion du chapitre

Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Chapitre 5 — Base de données

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Modèle conceptuel

Le noyau lie users aux fichiers, tokens, défis, dispositifs et événements. files conserve tailles, codec, empreinte, chemins internes, statut et preuve d'intégrité. share_codes relie fichier, émetteur et destinataire sans stocker le code en clair. audit_events et auth_events séparent l'audit fonctionnel de la chronologie d'authentification.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Contraintes et données sensibles

Les e-mails, identifiants de dispositif et hachages de tokens sont indexés ou uniques selon leur rôle.

Les secrets TOTP sont sensibles et ne doivent jamais apparaître dans les logs; le code actuel nécessite leur disponibilité serveur pour valider le facteur, sans preuve d'un chiffrement applicatif au repos.

Les chemins d'artefacts sont internes et ne constituent pas des URL publiques.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Transactions et migrations

Une opération métier doit conserver la cohérence entre fichier physique et ligne SQL.

Les migrations versionnent l'évolution, mais un volume persistant Docker peut conserver un schéma ancien; le runner doit donc s'exécuter et échouer explicitement.

La sauvegarde, la rétention et la purge ne sont pas établies par les modèles et demeurent des décisions d'exploitation.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Tableau 5 — Dictionnaire de données

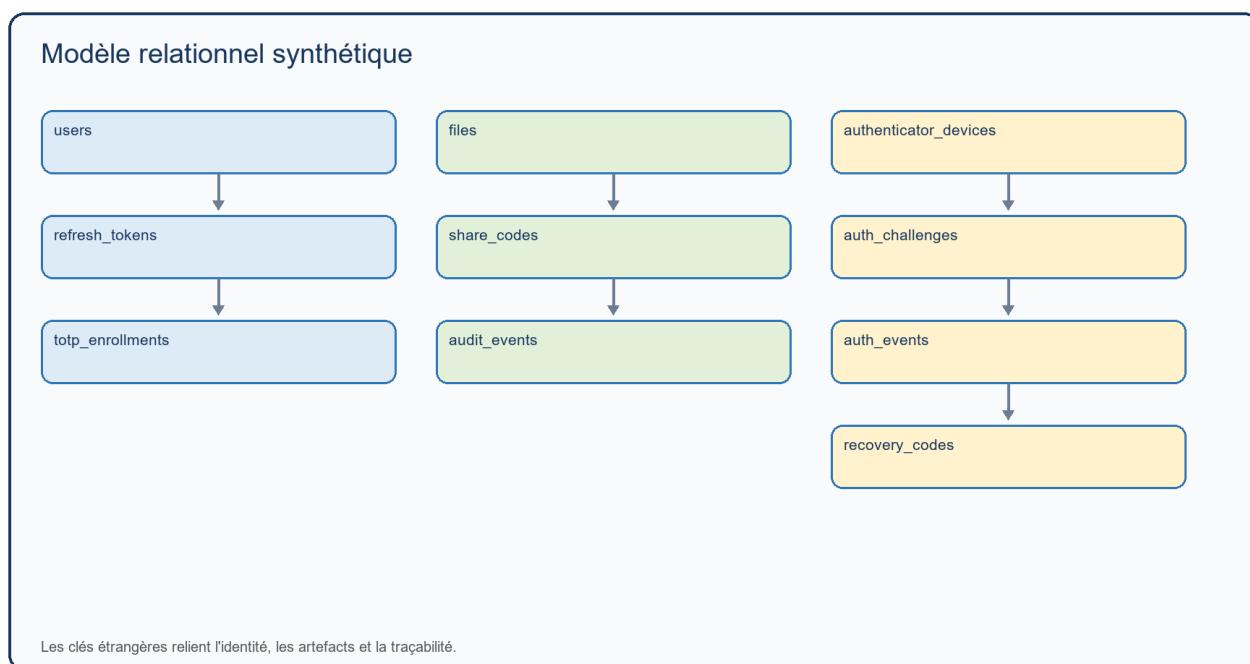
Table	Finalité	Relations
users	Identité, rôle, MFA	Parent de fichiers, tokens et dispositifs
totp_enrollments	Enrôlement borné	user_id
files	Artefacts et métriques	owner_id
share_codes	Partage destinataire	file_id, sender_id
audit_events	Traçabilité métier	user_id logique
account_verification_challenges	Vérification/récupération	user_id
refresh_tokens	Renouvellement révoquable	user_id
authenticator_devices	Dispositifs enregistrés	user_id
auth_challenges	Défis push/number match	user_id, device_id
auth_events	Historique authentification	user_id, device_id
recovery_codes	Récupération MFA	user_id
auth_policies	Politique globale	scope unique

Interprétation — La synthèse est circonscrite aux preuves indiquées.

Conclusion du chapitre

Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Figure 5 — Erd



Interprétation — Le diagramme matérialise les responsabilités, dépendances et frontières où les contrôles doivent intervenir.

Chapitre 6 — Moteur de compression IA

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Architecture hybride

Hybrid V3 ne remplace pas les codecs classiques: il les orchestre.

Les caractéristiques du fichier alimentent Selector V2; la stratégie top-3 évalue un ensemble borné de candidats puis sérialise le meilleur choix dans XAIC v6.

Le sélecteur est gelé dans checkpoints/selector_v2/best.json; aucun artefact Selector V3 n'est revendiqué.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

CausalByteGRU

Le CausalByteGRU transforme les octets en embeddings, propage un état récurrent et produit des logits sur 256 symboles.

Les probabilités doivent être déterministes pour que le décodeur reproduise la même distribution.

La porte de mise à jour s'écrit $z_t = \text{sigmoid}(W_z x_t + U_z h_{t-1})$.

La porte de réinitialisation s'écrit $r_t = \text{sigmoid}(W_r x_t + U_r h_{t-1})$.

L'état candidat combine x_t et le produit élément par élément r_t fois h_{t-1} ; l'état final interpole l'ancien état et le candidat selon z_t .

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Format, intégrité et compatibilité

Le conteneur enregistre version, stratégie, métadonnées nécessaires et checksums.

La compatibilité couvre XAIC v1 à v6 dans les tests historiques du rapport V3.

La condition essentielle n'est pas seulement l'ouverture du fichier mais l'égalité SHA-256 entre origine et reconstruction.

Un checkpoint absent ou incompatible doit être une erreur explicite, jamais une reconstruction approximative.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Mesures

Le taux $CR = \text{taille_originale} / \text{taille_compressée}$; l'économie vaut $(1 - \text{taille_compressée} / \text{taille_originale}) \times 100$; $BPB = 8 \times \text{taille_compressée} / \text{taille_originale}$.

Le débit divise les octets par le temps complet.

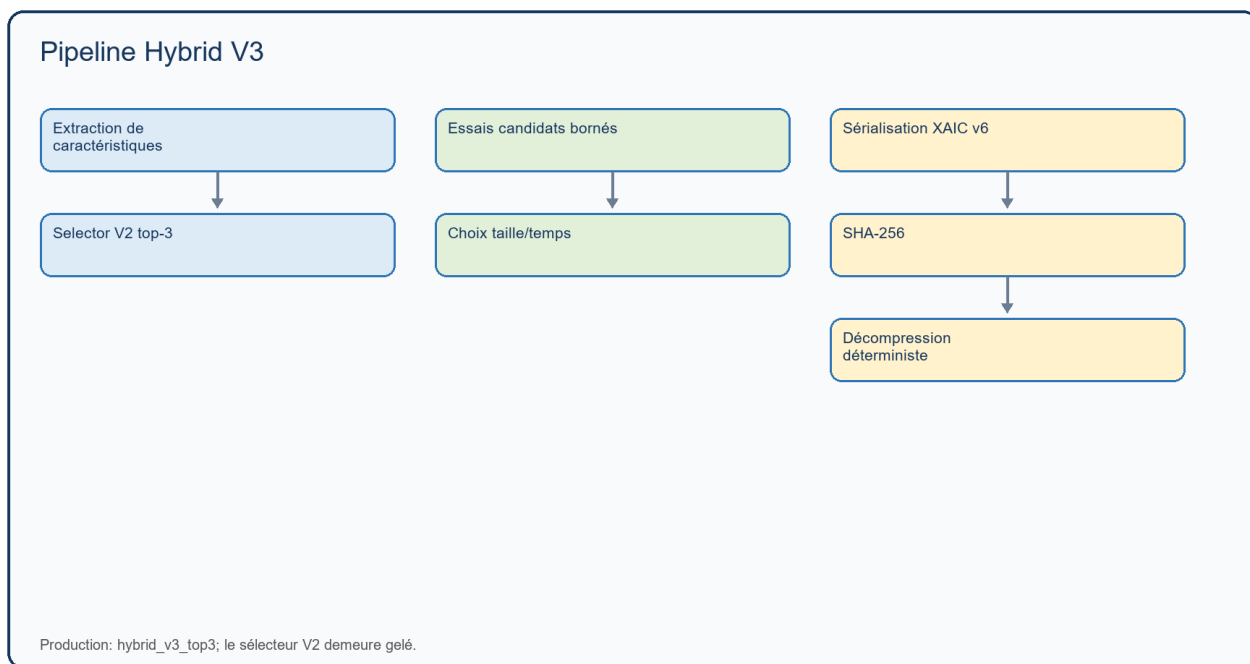
Le protocole V3 inclut sélection, compression, sérialisation, checksums et round-trip, ce qui évite de présenter une micro-mesure de codec comme temps utilisateur.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Conclusion du chapitre

Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Figure 6 — Compression pipeline



Interprétation — Le diagramme matérialise les responsabilités, dépendances et frontières où les contrôles doivent intervenir.

Chapitre 7 — Méthodologie d'entraînement

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Corpus et fenêtrage

Les scripts construisent des corpus par familles, excluent les artefacts impropres, bornent les octets par fichier et produisent des fenêtres causales.

Plusieurs configurations existent: contextes 64, 128, 256 et recherche 512; le checkpoint Kaggle documente contexte 256, stride 128, maximum 32 MiB par fichier et seed 42.

Ces valeurs sont attachées aux artefacts concernés et non généralisées à tous les modèles.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Optimisation

La tâche minimise la cross-entropie du prochain octet.

Les historiques montrent notamment un apprentissage à 0,001 puis décroissance pour le checkpoint Kaggle, et 0,0001 dans l'historique neural_lossless_v2.

DataLoader, batch, découpage et échantillonnage doivent conserver l'étanchéité train/validation.

Une loss favorable justifie la modélisation, mais seule l'encodeur réel établit les BPB d'artefact.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Sélection et reproductibilité

Le seed, le manifeste de corpus, l'empreinte du checkpoint, la configuration et l'environnement doivent être conservés ensemble.

Le Selector V2 atteint top-1 0,666667, top-2 0,810753 et top-3 0,890323 dans son rapport; sa latence d'inférence indiquée est 0,325453 ms.

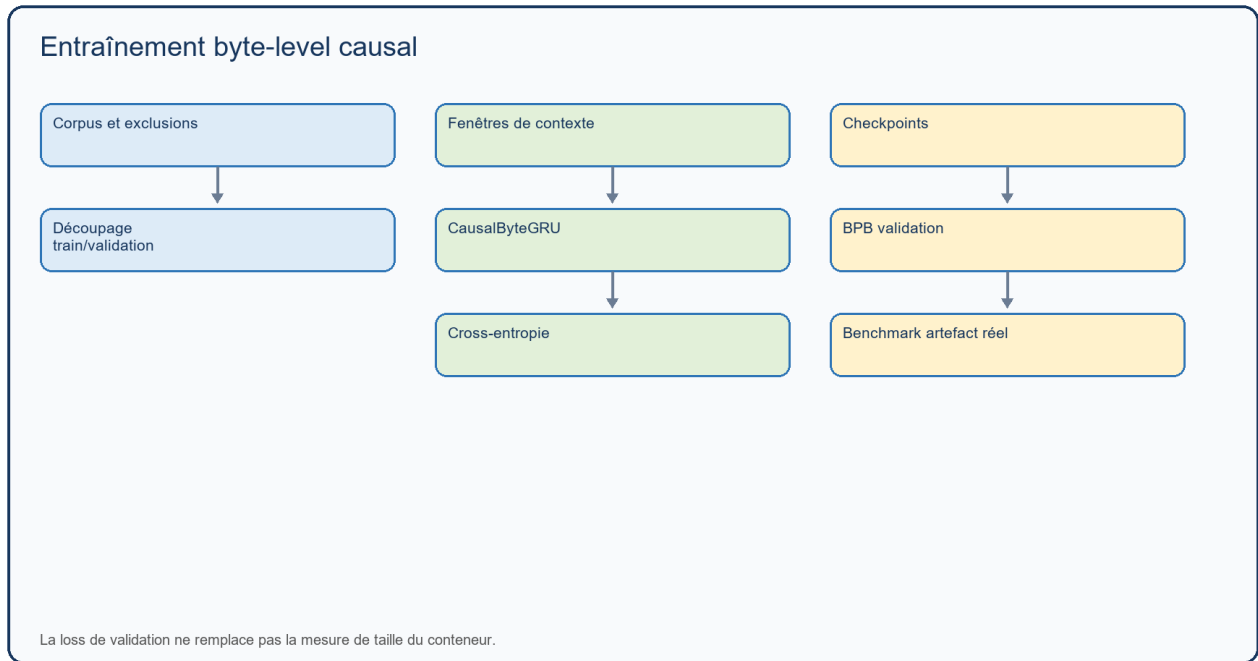
Ces métriques de classement n'impliquent pas que la taille finale bat le meilleur codec fixe.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Conclusion du chapitre

Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Figure 7 — Training pipeline



Interprétation — Le diagramme matérialise les responsabilités, dépendances et frontières où les contrôles doivent intervenir.

Chapitre 8 — Architecture de sécurité

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Défense en profondeur

Le mot de passe est haché; les tokens persistants et codes sont hachés; les JWT expirent; TOTP ajoute une preuve temporelle; l'autorisation vérifie propriétaire ou rôle; SHA-256 protège l'intégrité; ClamAV et YARA recherchent respectivement signatures antimalware et motifs; Docker réduit les couplages.

Chaque mécanisme couvre une menace différente et aucun ne prouve seul la sûreté globale.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Pipelines fail-closed

Si ClamAV est indisponible, si les règles YARA sont invalides ou si la limite de sortie est dépassée, le traitement est rejeté.

Les tests ciblent ces conditions et la non-invocation du moteur après blocage.

La décompression applique des bornes avant parsing coûteux, traite en espace temporaire contrôlé et n'expose le résultat qu'après scan.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Menaces et risques résiduels

Les menaces principales sont malware, bombe de décompression, vol de token, bruteforce, rejeu, énumération de compte, client compromis, règles mal configurées et secrets au repos.

Les risques résiduels comprennent l'absence de preuve opérationnelle permanente des scanners, la conservation locale de tokens du portail, l'absence de politique de rétention finalisée et la dépendance aux fournisseurs e-mail.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Tableau 6 — Menaces

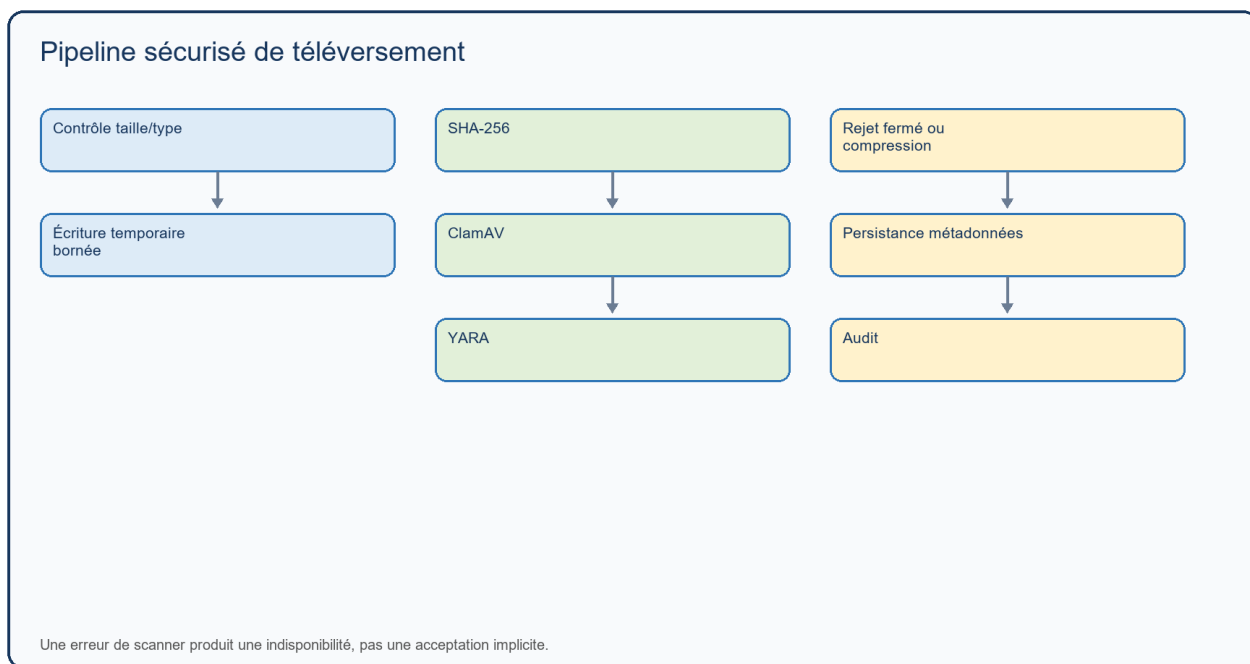
Menace	Vecteur	Impact	Contrôle	Risque résiduel
Malware	Upload	Élevé	ClamAV + YARA	Signatures/daemon
Bombe	XAIC forgé	Élevé	Bornes pré-parsing	Codec nouveau
Vol token	Client compromis	Élevé	Expiration + révocation refresh	Access token vivant
Bruteforce	Login/TOTP	Élevé	Compteurs + expiration	Distribution attaque
Rejeu	Code ou défi	Moyen	Nonce/usage unique	Horloge/implémentation
Règle invalide	Mauvaise config	Élevé	Fail-closed	Indisponibilité

Interprétation — La synthèse est circonscrite aux preuves indiquées.

Conclusion du chapitre

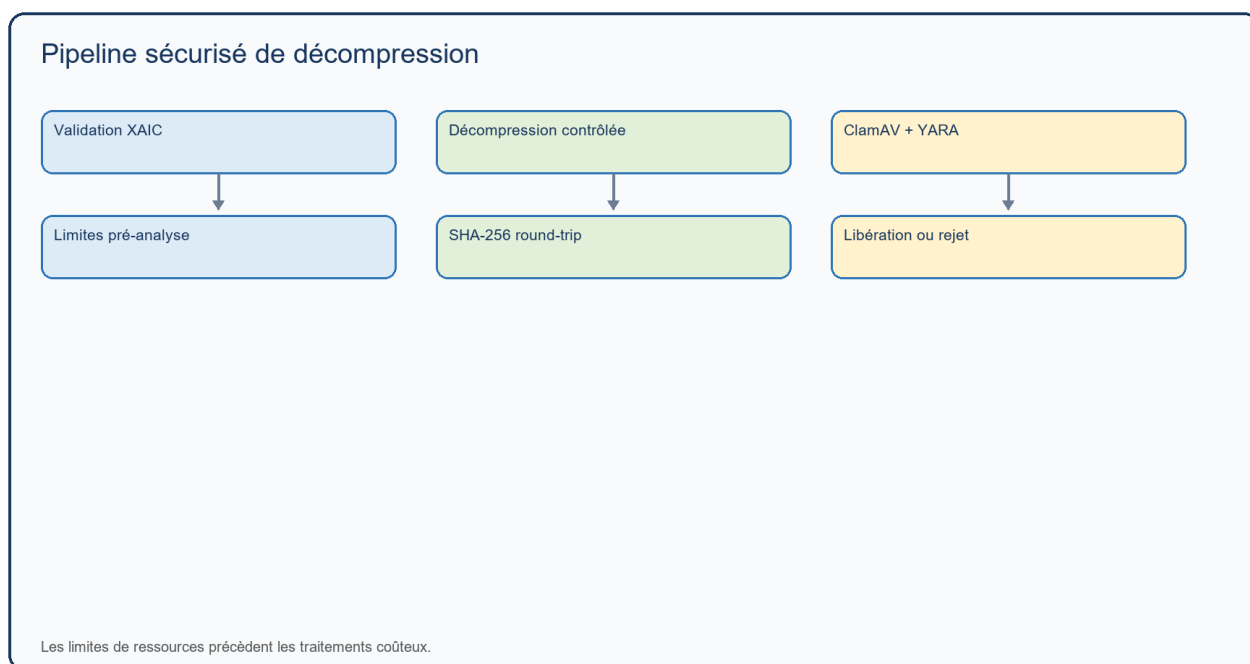
Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Figure 8 — Upload pipeline



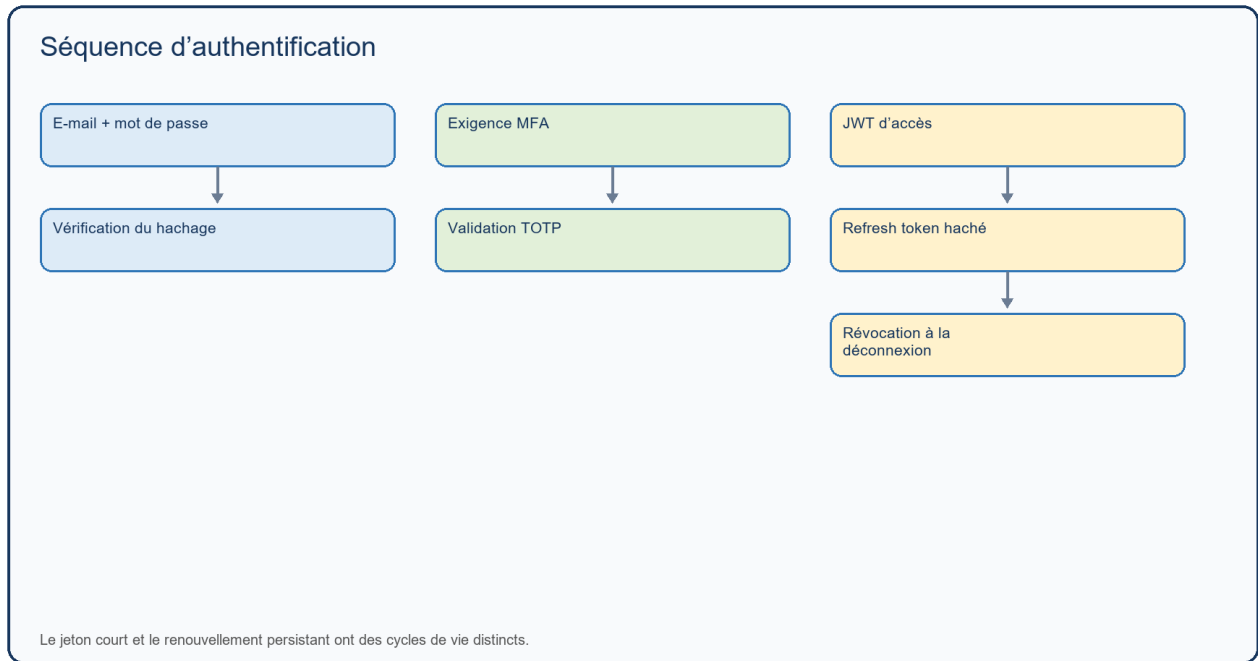
Interprétation — Le diagramme matérialise les responsabilités, dépendances et frontières où les contrôles doivent intervenir.

Figure 9 — Decompression pipeline



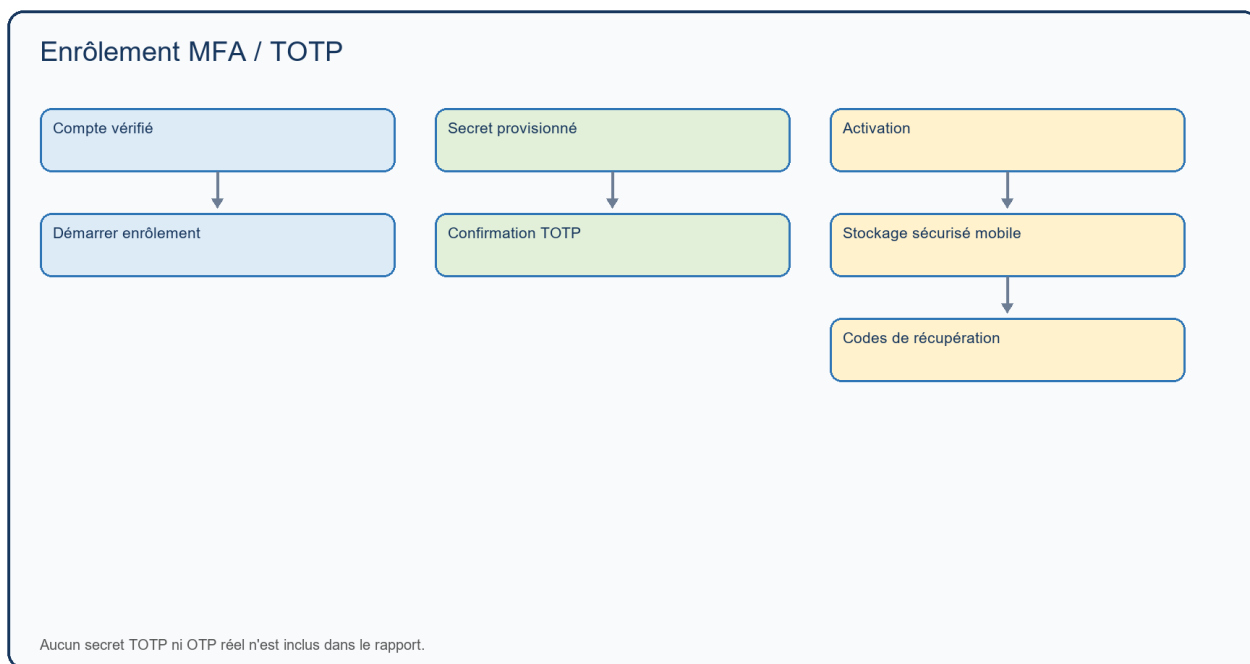
Interprétation — Le diagramme matérialise les responsabilités, dépendances et frontières où les contrôles doivent intervenir.

Figure 10 — Auth sequence



Interprétation — Le diagramme matérialise les responsabilités, dépendances et frontières où les contrôles doivent intervenir.

Figure 11 — Mfa sequence



Interprétation — Le diagramme matérialise les responsabilités, dépendances et frontières où les contrôles doivent intervenir.

Chapitre 9 — Authentificateur mobile Flutter

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Rôle et états

L'application mobile ne compresse pas: elle guide inscription, vérification d'e-mail, enrôlement TOTP, confirmation serveur, verrouillage local et affichage des codes rotatifs.

Le cycle observable comprend démarrage/chargement, non authentifié, vérification, enrôlement, authentifié et verrouillé; les transitions échouent fermées lorsque le stockage ou la biométrie ne sont pas disponibles.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Stockage et biométrie

Le matériel d'authentification est confié au secure storage de la plate-forme.

La biométrie protège l'accès local mais ne remplace pas la validation serveur.

Un correctif audité refuse désormais le déverrouillage lorsque l'authentification de l'appareil est non supportée.

Les OTP, secrets et QR réels doivent être masqués dans toute capture.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Configuration réseau

deployment_config.dart sépare les URL de développement et de production.

Sur appareil Android physique, localhost vise le téléphone et non le poste; l'API de développement doit être joignable sur le LAN et déclarée selon la politique réseau Android.

En production, HTTPS et une URL fixe vérifiée sont requis.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Conclusion du chapitre

Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Chapitre 10 — Application Desktop Flutter

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Architecture

Le desktop propose les modes local et cloud.

ApiService gère l'API, LocalEngineService lance le moteur local, HistoryService persiste l'historique, SettingsService conserve la configuration et AppState orchestre session et vues.

Cette séparation permet des tests de sérialisation, session et modèles sans exécuter tout le moteur.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Parcours

Les écrans compress, decompress, history, receive et settings couvrent sélection, options, résultats, erreurs, téléchargements et réception de partage.

Le chemin cloud dépend du scanner et de l'identité; le chemin local dépend de Python, du package et des checkpoints.

Les métriques affichées doivent provenir de mesures persistées, jamais d'une animation de progression.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Limites

L'annulation cloud n'est pas annoncée si aucun endpoint ne la supporte.

L'intégrité n'est déclarée vérifiée que si la preuve existe.

Les courses d'annulation d'un processus local, les chemins externes et l'installation propre restent à tester sur une distribution signée.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Conclusion du chapitre

Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Chapitre 11 — Portail Angular

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Architecture front-end

Les composants couvrent authentification, tableau de bord, compression, décompression, historique, partage, sécurité et paramètres.

Les services centralisent API et session; l'intercepteur sérialise le refresh pour éviter plusieurs renouvellements concurrents.

Une erreur irrécupérable efface la session expirée.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

UX et sécurité

Les routes protégées nécessitent un guard.

Le portail n'effectue pas d'autorisation définitive: il masque pour l'ergonomie tandis que l'API décide.

Les téléchargements initient une sauvegarde navigateur sans constituer une preuve de persistance disque.

Les erreurs de scanner indiquent une indisponibilité de traitement, non un faux succès.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Validation

Vingt-quatre définitions de tests TypeScript ont été recensées, mais l'audit produit signale qu'aucune cible de test Angular active n'était configurée.

Il faut donc distinguer l'existence des specs de leur exécution actuelle et conserver la validation navigateur E2E comme exigence avant livraison.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Conclusion du chapitre

Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Chapitre 12 — Site public Next.js

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Positionnement

Le site public explique le produit, oriente vers le portail, expose l'état et la distribution.

Il ne doit pas simuler des performances, des tarifs ou une disponibilité.

Le manifeste de versions détermine les boutons de téléchargement; en l'absence d'artefact signé, les actions restent désactivées.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

État et partage public

La page d'état consomme /public/status et ne déduit pas la santé complète d'un simple /health.

La page de partage public révèle seulement qu'une authentification est nécessaire; nom, taille et codec restent derrière l'autorisation du destinataire. robots et noindex limitent l'indexation de ces pages sensibles.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Limites de publication

L'audit récent a observé du contenu déployé ancien et des routes manquantes, sans autoriser ce rapport à affirmer qu'elles sont corrigées en production.

La documentation publique et les politiques juridiques restent incomplètes; les claims de conformité, zero-knowledge ou chiffrement de bout en bout ne sont pas établis.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Conclusion du chapitre

Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Chapitre 13 — Administration .NET

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Architecture Blazor

L'administration Blazor Server conserve les tokens dans le circuit serveur, vérifie /admin/stats après connexion et efface la session sur 401/403.

Le tableau de bord lit des données API: utilisateurs, fichiers, audit, état scanner et configuration e-mail assainie.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Fonctions réellement disponibles

Les vues utilisateurs et jobs sont en lecture; l'audit est borné; système et scanner exposent un état.

Incidents, quarantaine, suspension, changement de rôle et suppression ne disposent pas de modèles/API complets et sont présentés comme indisponibles.

Cette retenue évite une interface d'administration fictive ou dangereuse.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Contrôles

Le rôle admin est vérifié côté API.

Le logout tente de révoquer le refresh token mais ne transforme pas instantanément un access token déjà émis en jeton invalide.

Pagination, tests .NET dédiés et session navigateur authentifiée constituent des lacunes.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Conclusion du chapitre

Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Chapitre 14 — E-mail et cycle de compte

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Providers

Le service distingue SMTP, Mailpit en local, Brevo et Resend selon configuration.

Mailpit capture les messages sans livraison externe et facilite les tests.

En production, l'expéditeur, le domaine et les secrets appartiennent au fournisseur et doivent être vérifiés hors du dépôt.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Cycle

Inscription, code e-mail, activation, connexion, MFA, session et récupération sont des états reliés, pas des écrans isolés.

Les réponses d'oubli de mot de passe doivent rester suffisamment uniformes pour limiter l'énumération.

Les codes ont expiration, compteur de tentatives et usage unique; les journaux ne doivent contenir ni code ni jeton.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Pièces jointes et limites

Les capacités e-mail sont exposées par endpoint afin que le client ne suppose pas le support d'une pièce jointe.

Le succès HTTP d'un fournisseur ne prouve pas la remise dans la boîte du destinataire; la validation finale requiert réception réelle et analyse des rebonds.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Conclusion du chapitre

Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Chapitre 15 — Conteneurisation et DevOps

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Docker Compose

Compose orchestre API, PostgreSQL, ClamAV et Mailpit, avec réseaux, volumes, variables et health checks.

Les données PostgreSQL persistantes survivent au conteneur; les migrations restent donc obligatoires.

Les images PyTorch peuvent être lourdes et augmentent temps de build et surface de dépendances.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Environnements

Le développement accepte des origines et hôtes locaux explicitement déclarés; la production exige secrets forts, HTTPS, scanners réels et comptes administrateur provisionnés.

Les fichiers .env sont des entrées sensibles et le rapport ne lit ni ne reproduit leurs valeurs.

Les configurations Vercel/Render constituent des intentions vérifiables, pas une attestation de mise en ligne.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

CI/CD et exploitation

La présence de workflows et checklists facilite lint, tests et builds, mais les gates finales incluent signature des applications, migrations, scan réel, round-trip et smoke tests authentifiés.

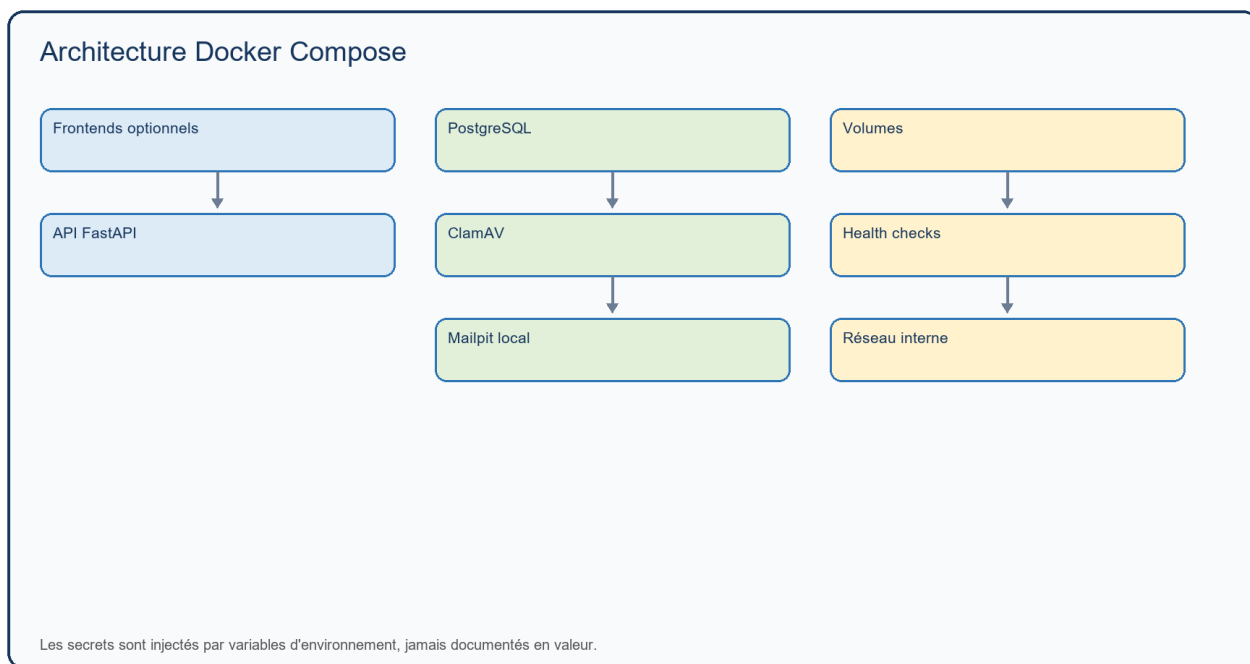
Kubernetes, Vault et monitoring apparaissent comme infrastructure préparatoire; leur présence ne prouve pas un cluster exploité.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Conclusion du chapitre

Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Figure 12 — Docker architecture



Interprétation — Le diagramme matérialise les responsabilités, dépendances et frontières où les contrôles doivent intervenir.

Chapitre 16 — Tests et assurance qualité

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Stratégie

La pyramide combine tests unitaires de fonctions pures, intégration SQL/API, contrats croisés, scanners simulés et tests moteur.

L'inventaire actuel compte 124 définitions backend, 146 moteur, 36 desktop, 24 Angular et 12 mobile.

Ces nombres décrivent le source; les résultats historiques documentent 93 backend, 15 mobile, 5 desktop avec un live test ignoré, et 207 Python plus 3 ignorés pour V3.

Ils ne sont pas mélangés.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Sécurité et régression

Les scénarios couvrent scanner absent, règle YARA invalide, origine CORS, limites avant parsing, erreur assainie, upload bloqué sans compression, partage expiré/révoqué, migrations et MFA.

Un test mocké valide la logique mais pas le daemon, la DLL, la connectivité ou la fraîcheur des signatures en production.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Critères de sortie

Une release doit produire builds propres, round-trip SHA, scanners opérationnels, migrations appliquées, e-mail reçu, MFA sur téléphone réel, installation signée et navigation multi-écran.

Toute preuve manquante reste BLOCKED plutôt qu'inférée.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Tableau 7 — Inventaire des tests

Composant	Définitions recensées	Dernière preuve d'exécution distincte
FastAPI	124	93 réussis (audit produit)
Moteur Python	146	207 réussis, 3 ignorés (rapport V3)
Rust	2 cas historiques	2 réussis (rapport V3)
Flutter Desktop	36	5 réussis, 1 live ignoré
Flutter Mobile	12	15 réussis (audit produit)
Angular	24	Specs présentes; cible active non établie

Interprétation — La synthèse est circonscrite aux preuves indiquées.

Conclusion du chapitre

Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Chapitre 17 — Performances et résultats

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Protocole V3

Le benchmark corrigé utilise 140 fichiers held-out uniques, 368 446 658 octets originaux et une comparaison appariée.

Les temps incluent sélection, compression, conteneur, checksums et décompression.

Trois répétitions sont utilisées jusqu'à 4 MiB, une au-delà; Brotli-11 n'a qu'une répétition car son coût est prohibitif.

Les limites doivent accompagner toute généralisation.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Résultats

Hybrid V3 produit 362 188 760 octets, 7,864124 BPB, 18,027349 MiB/s en compression et 32,202720 MiB/s en décompression.

Brotli-11 produit 359 459 933 octets, 7,804873 BPB, 0,251334 MiB/s et 194,827433 MiB/s.

V3 est 0,7591 % plus grand, mais son débit de compression est environ 70,7 fois celui de Brotli-11 selon le rapport.

Les 140 fichiers sont tous perdus en taille face à Brotli-11, ce qui interdit une prétention de meilleur ratio.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Interprétation

V3 améliore V2 de 1,0139 % en taille et de 220,2570 % en vitesse; il est proche de V1 en taille (+0,1318 %) tout en étant nettement plus rapide.

La valeur produit dépend donc de la pondération latence/stockage.

La décompression Brotli demeure beaucoup plus rapide; le choix top-3 est pertinent pour l'encodage interactif, pas automatiquement pour l'archivage minimal.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Tableau 8 — Résultats Hybrid V3

Méthode	Octets compressés	BPB	Comp. MiB/s	Décomp. MiB/s
Hybrid V1	361 712 014	7,853772	2,816161	20,174976
Hybrid V2 gelé	365 898 448	7,944671	5,629025	30,124013
Hybrid V3 top-3	362 188 760	7,864124	18,027349	32,202720
Brotli-11	359 459 933	7,804873	0,251334	194,827433

Interprétation — La synthèse est circonscrite aux preuves indiquées.

Conclusion du chapitre

Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Chapitre 18 — Défis, limites et perspectives

Introduction du chapitre. Cette partie confronte les concepts aux éléments réellement observés et conserve les limites de preuve.

Défis d'ingénierie

Les difficultés étayées incluent configuration API par plate-forme, Android sur LAN, persistance des migrations Docker, alignement `audit_events.updated_at`, providers e-mail, cycle MFA, fail-closed scanner, performance neuronale et poids PyTorch.

Le diagnostic a consisté à isoler frontière, reproduire par test, corriger sans affaiblir le contrôle puis ajouter une régression.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Limites actuelles

La production n'est pas attestée de bout en bout; l'authentificateur n'est pas validé sur téléphone signé; Angular ne possède pas une cible de test active; l'administration n'a pas de projet de tests; la rétention et la suppression de compte ne sont pas implémentées; les secrets TOTP n'ont pas de preuve de chiffrement applicatif au repos; Brotli-11 reste plus compact; l'explicabilité est une traçabilité de sélection, non une explication neuronale fine.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Perspectives

Les travaux futurs prioritaires sont la quantification et le profiling CPU, un sélecteur calibré coût/temps, des explications par caractéristiques, object storage, jobs distribués, observabilité, scans isolés, rotation de clés, stockage matériel mobile, CI/CD signé et dataset versionné.

Kubernetes n'est justifié qu'après mesure de charge; la complexité ne doit pas précéder le besoin.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Conclusion générale

XAI-Compress démontre une architecture multi-client intégrant compression adaptative, contrôle de contenu, identité forte et preuves expérimentales.

Sa contribution la plus solide est la cohérence entre sélection hybride, conteneur déterministe, SHA-256 et défense en profondeur.

Les résultats V3 révèlent un compromis clair: forte accélération de compression par rapport à Brotli-11, au prix d'un léger surcoût de taille.

Le projet atteint un niveau de prototype d'ingénierie riche et testable, mais la livraison industrielle reste conditionnée par les validations réelles, la signature, la politique de données et l'exploitation continue des scanners.

Lecture d'ingénierie. La décision est évaluée selon l'exactitude fonctionnelle, la frontière de confiance, le coût opérationnel et la preuve de validation. Une configuration n'est pas assimilée à une exécution de production.

Conclusion du chapitre

Les résultats établis réduisent l'écart entre système annoncé et système démontré. Les limites alimentent les critères de validation et les perspectives.

Références bibliographiques

- [1] RFC 1951 — DEFLATE Compressed Data Format Specification, IETF, 1996.
- [2] RFC 7519 — JSON Web Token (JWT), IETF, 2015.
- [3] RFC 6238 — TOTP: Time-Based One-Time Password Algorithm, IETF, 2011.
- [4] Cho et al., Learning Phrase Representations using RNN Encoder–Decoder, EMNLP, 2014.
- [5] OWASP, Authentication Cheat Sheet et File Upload Cheat Sheet, documentation officielle.
- [6] FastAPI, Security and Dependencies, documentation officielle.
- [7] Flutter, Secure storage and platform integration, documentation officielle.
- [8] Angular, Routing, guards and HTTP interceptors, documentation officielle.
- [9] Next.js, Deployment and security documentation, documentation officielle.
- [10] PostgreSQL, Constraints, transactions and indexes, documentation officielle.
- [11] Docker, Compose specification and secrets, documentation officielle.
- [12] ClamAV, clamd and signature database documentation, documentation officielle.
- [13] VirusTotal, YARA documentation, documentation officielle.
- [14] PyTorch, GRU and reproducibility documentation, documentation officielle.
- [15] Google, Brotli format and implementation documentation.

Annexe A — Inventaire consolidé des endpoints

Méthode	Endpoint	Contrôle	Finalité
GET	/health	Public	Vivacité
POST	/auth/register	Public	Inscription
POST	/auth/login	Public + MFA	Connexion
POST	/auth/refresh	Refresh	Renouvellement
POST	/auth/logout	Refresh	Révocation
GET	/auth/me	JWT	Profil
POST	/auth/verification/email/send	JWT	Envoyer code e-mail
POST	/auth/verification/email/confirm	JWT	Confirmer e-mail
POST	/auth/verification/phone/send	JWT	Envoyer code téléphone
POST	/auth/verification/phone/confirm	JWT	Confirmer téléphone
POST	/auth/password/forgot	Public	Initier récupération
POST	/auth/password/verify-code	Public	Vérifier code
POST	/auth/password/reset	Jeton reset	Changer mot de passe
GET	/auth/totp/status	JWT	État TOTP
POST	/auth/totp/enroll	JWT	Démarrer TOTP
POST	/auth/totp/resent	JWT	Renvoyer preuve
POST	/auth/totp/email/confirm	JWT	Confirmer canal
POST	/auth/totp/confirm	JWT	Activer TOTP
POST	/auth/authenticator/enroll/start	JWT	Démarrer appareil
POST	/auth/authenticator/enroll/confirm	JWT	Confirmer appareil
POST	/auth/devices	JWT	Enregistrer dispositif
GET	/auth/devices	JWT	Lister dispositifs
POST	/auth/devices/{id}/revoke	JWT	Révoquer dispositif
POST	/auth/challenges	JWT	Créer défi
GET	/auth/challenges	JWT	Lister défis
POST	/auth/challenges/{id}/approve	JWT + dispositif	Approuver
POST	/auth/challenges/{id}/reject	JWT + dispositif	Rejeter
GET	/auth/history	JWT	Historique auth
POST	/auth/recovery-codes	JWT + MFA	Codes de secours
POST	/files	JWT	Créer métadonnées
POST	/compression/jobs	JWT + scanners	Compresser
GET	/files/{id}/download	JWT + propriétaire	Télécharger
POST	/compression/decompress	JWT + scanners	Décompresser
POST	/files/{id}/email	JWT + propriétaire	Envoyer artefact
GET	/email/capabilities	JWT	Capacités e-mail
GET	/history	JWT	Historique
POST	/shares	JWT + propriétaire	Créer partage
GET	/shares	JWT	Lister partages
POST	/shares/{id}/revoke	JWT + propriétaire	Révoquer
POST	/shares/redeem	JWT destinataire	Racheter
GET	/public/shares/{code}	Public assaini	Disponibilité partage
POST	/shares/download	JWT destinataire	Télécharger partage

Méthode	Endpoint	Contrôle	Finalité
GET	/admin/stats	Admin	Statistiques
GET	/admin/users	Admin	Utilisateurs
GET	/admin/jobs	Admin	Fichiers/jobs
GET	/admin/users/{id}	Admin	Détail utilisateur
GET	/admin/security/scanner	Admin	État scanners
GET	/admin/audit	Admin	Audit
GET	/admin/email/configuration	Admin	Configuration assainie
GET	/public/status	Public assaini	État services

Annexe B — Preuves principales

43 routes REST observées

Preuve: services/api_fastapi/app/main.py; account.py; mfa.py; authenticator.py

Confiance: VERIFIED FROM SOURCE

12 tables SQLAlchemy

Preuve: services/api_fastapi/app/models.py

Confiance: VERIFIED FROM SOURCE

Fail-closed ClamAV/YARA

Preuve: app/security_scanner.py; tests/test_security_scanner.py; test_product_truth.py

Confiance: VERIFIED FROM SOURCE/TEST

Refresh tokens hachés et révocables

Preuve: models.py; security.py; main.py

Confiance: VERIFIED FROM SOURCE

MFA TOTP et enrôlement

Preuve: mfa.py; authenticator.py; mobile lib/core/app_state.dart

Confiance: VERIFIED FROM SOURCE

Hybrid V3 top-3 retenu

Preuve: engines/XAI-Compress/results/hybrid_v3/final_report.md

Confiance: HISTORICAL AUTHORITATIVE ARTIFACT

140 fichiers, 368446658 octets

Preuve: engines/XAI-Compress/results/hybrid_v3/final_report.md

Confiance: HISTORICAL AUTHORITATIVE ARTIFACT

Brotli-11 plus compact que V3

Preuve: engines/XAI-Compress/results/hybrid_v3/final_report.md

Confiance: HISTORICAL AUTHORITATIVE ARTIFACT

Vercel + Render configurés

Preuve: deployment/README.md; app configs

Confiance: VERIFIED FROM CONFIG; LIVE NOT PROVEN

Mobile secure storage/biométrie

Preuve: mobile services/secure_account_store.dart; biometric_service.dart

Confiance: VERIFIED FROM SOURCE

Angular refresh sérialisé

Preuve: apps/web_angular source and product truth audit

Confiance: VERIFIED FROM SOURCE

Admin lecture utilisateurs/jobs/audit

Preuve: apps/admin_dotnet; API admin routes

Confiance: VERIFIED FROM SOURCE

Production prête

Preuve: Aucune preuve E2E complète

Confiance: NOT VERIFIED / DO NOT CLAIM

Annexe C — Figures expérimentales à compléter

Figure expérimentale E-1

[RÉSULTAT EXPÉRIMENTAL À INSÉRER] Loss entraînement: epoch/step, train_loss, learning_rate, checkpoint.

Données minimales attendues: identifiant de source, taille originale, méthode, répétition, temps complet, octets finaux, statut SHA-256 et environnement. La visualisation devra afficher la dispersion et non seulement une moyenne.

Figure expérimentale E-2

[RÉSULTAT EXPÉRIMENTAL À INSÉRER] Loss validation: epoch, validation_loss, validation_BPB et split.

Données minimales attendues: identifiant de source, taille originale, méthode, répétition, temps complet, octets finaux, statut SHA-256 et environnement. La visualisation devra afficher la dispersion et non seulement une moyenne.

Figure expérimentale E-3

[RÉSULTAT EXPÉRIMENTAL À INSÉRER] BPB par méthode sur les 140 source_id communs.

Données minimales attendues: identifiant de source, taille originale, méthode, répétition, temps complet, octets finaux, statut SHA-256 et environnement. La visualisation devra afficher la dispersion et non seulement une moyenne.

Figure expérimentale E-4

[RÉSULTAT EXPÉRIMENTAL À INSÉRER] Temps complet de compression et dispersion par taille.

Données minimales attendues: identifiant de source, taille originale, méthode, répétition, temps complet, octets finaux, statut SHA-256 et environnement. La visualisation devra afficher la dispersion et non seulement une moyenne.

Figure expérimentale E-5

[RÉSULTAT EXPÉRIMENTAL À INSÉRER] Temps complet de décompression par méthode.

Données minimales attendues: identifiant de source, taille originale, méthode, répétition, temps complet, octets finaux, statut SHA-256 et environnement. La visualisation devra afficher la dispersion et non seulement une moyenne.

Figure expérimentale E-6

[RÉSULTAT EXPÉRIMENTAL À INSÉRER] Top-k, regret et latence des sélecteurs avec intervalles.

Données minimales attendues: identifiant de source, taille originale, méthode, répétition, temps complet, octets finaux, statut SHA-256 et environnement. La visualisation devra afficher la dispersion et non seulement une moyenne.

Annexe D — Configuration sans valeurs

DATABASE_URL; JWT_SECRET; ACCESS_TOKEN_EXPIRE_MINUTES; REFRESH_TOKEN_EXPIRE_DAYS;
CORS_ORIGINS; STORAGE_ROOT; CLAMAV_HOST; CLAMAV_PORT; YARA_RULES_PATH;
EMAIL_PROVIDER; SMTP_HOST; SMTP_PORT; SMTP_USERNAME; SMTP_PASSWORD; BREVO_API_KEY;
RESEND_API_KEY; PUBLIC_API_URL.

Aucune valeur n'est reproduite. La présence d'un nom de variable ne prouve ni sa définition ni sa rotation en production.

Annexe E — Limites avant soumission

- Déploiement coordonné et round-trip authentifié non prouvés.
- MFA à valider sur téléphone physique avec e-mail réel.
- Applications à signer et installer proprement.
- Politique de rétention, suppression et confidentialité à finaliser.
- Courbes d'entraînement et environnement matériel du benchmark à insérer.
- Tests Angular exécutables et projet de tests Admin .NET à établir.

Annexe F — Porte de qualité finale

- ☐ Source et migrations alignées
- ☐ Aucun secret dans les artefacts
- ☐ SHA-256 round-trip
- ☐ ClamAV et YARA opérationnels
- ☐ MFA physique validé
- ☐ Builds signés
- ☐ Tests clients exécutés
- ☐ Captures assainies
- ☐ Références relues
- ☐ Champs institutionnels complétés

MOBILE-01 — Création de compte

À capturer: Ouvrir Inscription

Visible: Champs non remplis et bouton

À masquer: E-mail, mots de passe

[CAPTURE À INSÉRER — MOBILE-01]

Légende proposée — Création de compte. Chapitre 9. Priorité MANDATORY.

MOBILE-02 — Vérification e-mail

À capturer: Soumettre inscription

Visible: Écran code et renvoi

À masquer: Code et adresse réelle

[CAPTURE À INSÉRER — MOBILE-02]

Légende proposée — Vérification de l'adresse. Chapitre 9. Priorité MANDATORY.

MOBILE-03 — Enrôlement TOTP

À capturer: Après vérification

Visible: Instructions et statut

À masquer: QR, secret, OTP

[CAPTURE À INSÉRER — MOBILE-03]

Légende proposée — Enrôlement authentificateur. Chapitre 9. Priorité MANDATORY.

MOBILE-04 — Code circulaire

À capturer: Déverrouiller

Visible: Compte fictif et anneau

À masquer: OTP, e-mail

[CAPTURE À INSÉRER — MOBILE-04]

Légende proposée — Code TOTP rotatif. Chapitre 9. Priorité RECOMMENDED.

DESKTOP-01 — Compression cloud

À capturer: Connexion puis Compression

Visible: Fichier test, mode, statut

À masquer: Chemin personnel, token

[CAPTURE À INSÉRER — DESKTOP-01]

Légende proposée — Compression depuis Desktop. Chapitre 10. Priorité MANDATORY.

DESKTOP-02 — Résultat

À capturer: Terminer compression

Visible: Tailles, codec, SHA masqué

À masquer: Chemins, empreinte complète

[CAPTURE À INSÉRER — DESKTOP-02]

Légende proposée — Résultat de compression. Chapitre 10. Priorité MANDATORY.

ANGULAR-01 — Tableau de bord

À capturer: Connexion portail

Visible: Indicateurs et navigation

À masquer: E-mail personnel

[CAPTURE À INSÉRER — ANGULAR-01]

Légende proposée — Tableau de bord utilisateur. Chapitre 11. Priorité MANDATORY.

ANGULAR-02 — Compression

À capturer: Route /compress

Visible: Sélecteur, mode, action

À masquer: Token, fichier privé

[CAPTURE À INSÉRER — ANGULAR-02]

Légende proposée — Lancement de compression. Chapitre 11. Priorité MANDATORY.

ANGULAR-03 — Historique

À capturer: Route /history

Visible: Table et actions

À masquer: Noms privés

[CAPTURE À INSÉRER — ANGULAR-03]

Légende proposée — Historique des opérations. Chapitre 11. Priorité RECOMMENDED.

ADMIN-01 — Dashboard

À capturer: Connexion admin test

Visible: Mesures et état

À masquer: Comptes réels

[CAPTURE À INSÉRER — ADMIN-01]

Légende proposée — Tableau de bord administratif. Chapitre 13. Priorité MANDATORY.

ADMIN-02 — Audit

À capturer: Ouvrir Audit

Visible: Actions assainies

À masquer: Détails sensibles

[CAPTURE À INSÉRER — ADMIN-02]

Légende proposée — Journal d'audit. Chapitre 13. Priorité RECOMMENDED.

PUBLIC-01 — Accueil

À capturer: Ouvrir racine

Visible: Architecture et CTA

À masquer: Données démo obsolètes

[CAPTURE À INSÉRER — PUBLIC-01]

Légende proposée — Site public. Chapitre 12. Priorité RECOMMENDED.

TEST-01 — Tests backend

À capturer: Exécuter pytest ciblé

Visible: Résumé succès

À masquer: Chemins/variables

[CAPTURE À INSÉRER — TEST-01]

Légende proposée — Suite backend. Chapitre 16. Priorité MANDATORY.

TEST-02 — Tests moteur

À capturer: Exécuter pytest moteur

Visible: Résumé et skips

À masquer: Chemins personnels

[CAPTURE À INSÉRER — TEST-02]

Légende proposée — Suite moteur. Chapitre 16. Priorité MANDATORY.

TEST-03 — Services sains

À capturer: docker compose ps

Visible: Health states

À masquer: Secrets/env

[CAPTURE À INSÉRER — TEST-03]

Légende proposée — Services Docker. Chapitre 15. Priorité MANDATORY.

SEC-01 — Validation scanner

À capturer: Diagnostic contrôlé

Visible: ClamAV/YARA ready

À masquer: Règles privées

[CAPTURE À INSÉRER — SEC-01]

Légende proposée — Validation scanners. Chapitre 8. Priorité MANDATORY.

DEPLOY-01 — Déploiements

À capturer: Ouvrir dashboards

Visible: Services et dates

À masquer: IDs/tokens/logs

[CAPTURE À INSÉRER — DEPLOY-01]

Légende proposée — Déploiements déclarés. Chapitre 15. Priorité RECOMMENDED.